FINAL PROJECT SUBMISSION REPORT OF ELEVATE LABS CYBERSECURITY INTERNSHIP

NAME	**Jadav Dinesh**
Submitted to:	**Elevate Labs**
Name of the Academic Institute	**Ganpat University**

REPORT SUBMITTED TO

As part of the Cyber Security Internship, I have completed "Final Project : Personal Firewall using Python Report" by following all steps as instructed. Below is a detailed summary of each step followed during the task:

Final project: Personal Firewall using Python Report

Introduction

This project focuses on developing a lightweight personal firewall using Python. The firewall is designed to monitor network traffic, filter packets based on user-defined rules, and optionally interface with system-level firewalls such as iptables. It can also provide a GUI for real-time monitoring.

Abstract

The personal firewall is a Python-based application leveraging Scapy to sniff network packets. Users can define rules to block or allow specific IPs, ports, and protocols. Suspicious packets are logged for auditing purposes. The project demonstrates core concepts in network security and offers both CLI and GUI interfaces for flexibility.

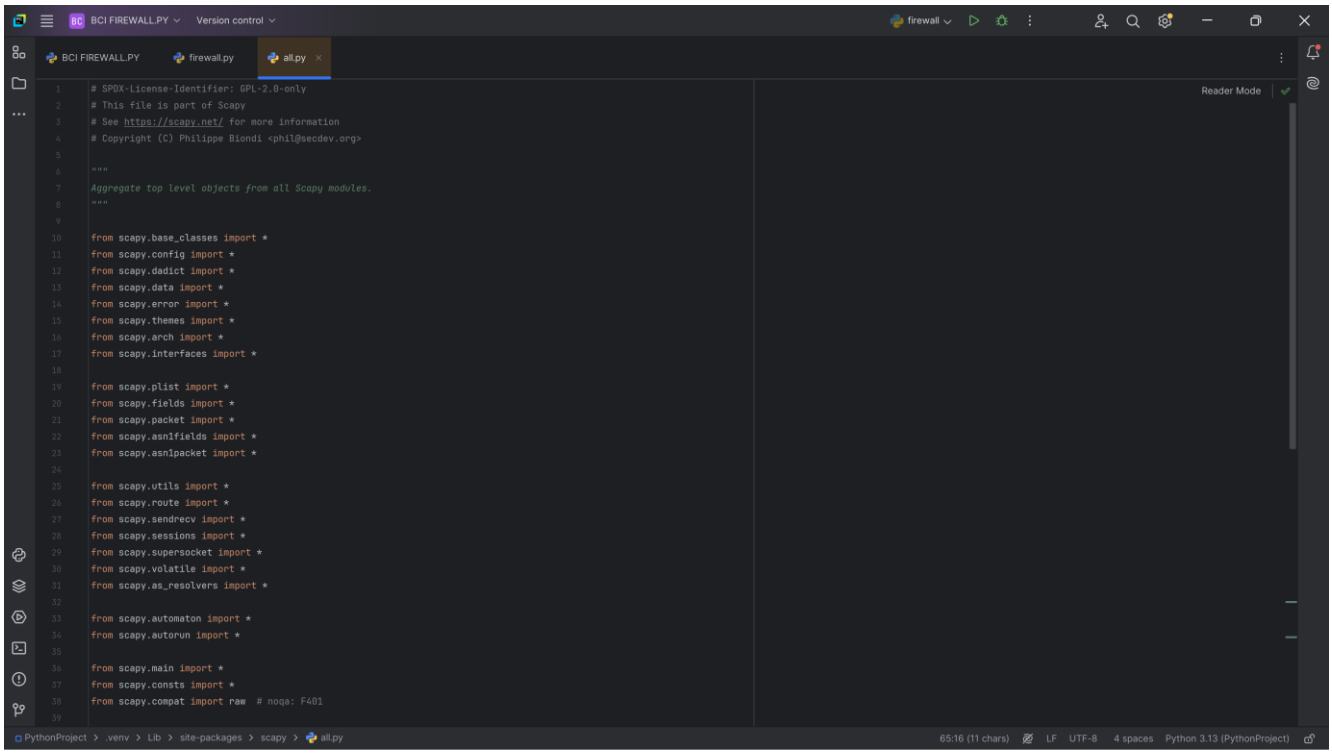
Tools Used

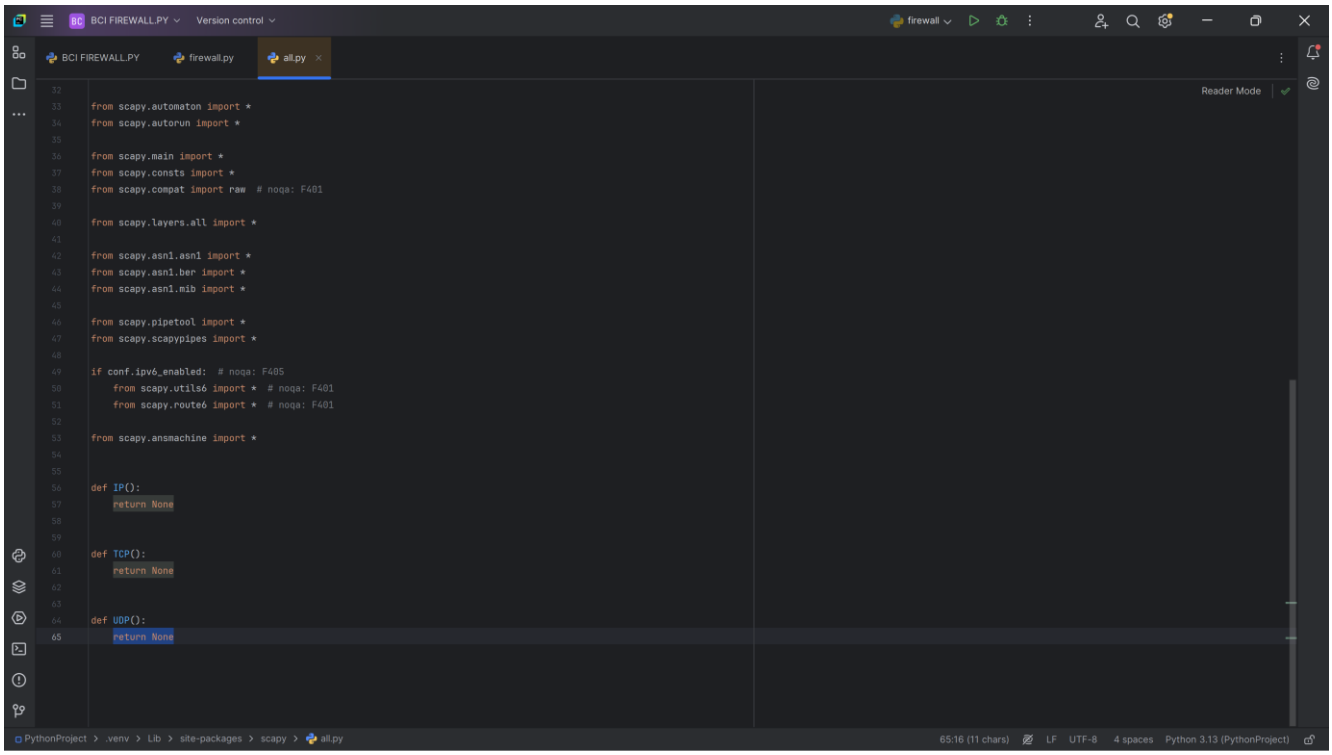
- **Python 3.13.3**
- **Scapy (for packet sniffing)**
- **iptables (for system-level enforcement, Linux)**
- **Tkinter (for optional GUI)**

Steps Involved in Building the Project

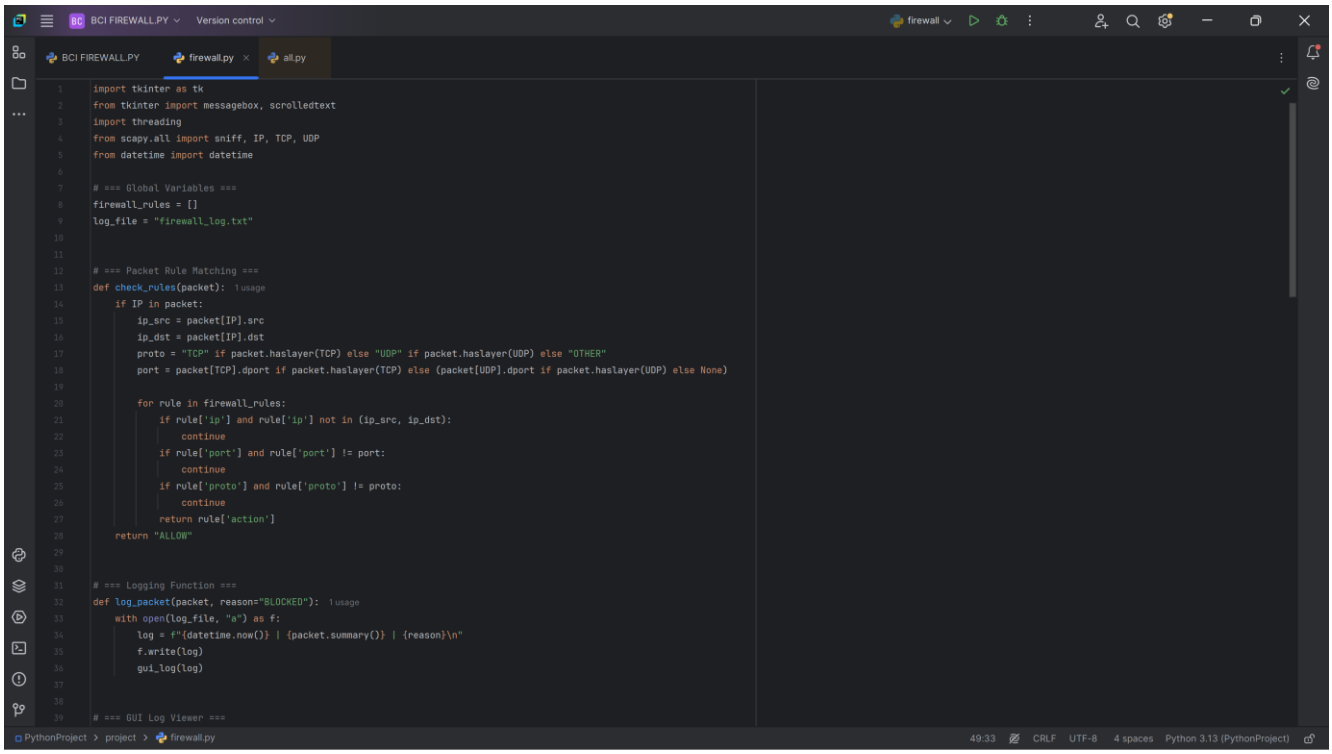
1. **Initialized packet sniffing using Scapy to capture live traffic.**
2. **Designed a rule-matching system to filter packets by IP, port, and protocol.**
3. **Implemented logging of blocked packets into a text file.**
4. **Developed a Tkinter GUI to allow rule management and display live logs.**
5. **Enabled optional integration with iptables for advanced blocking at the OS level**
6. **Tested the firewall against various IP traffic scenarios in a controlled environment.**

Final project: Personal Firewall using Python Report

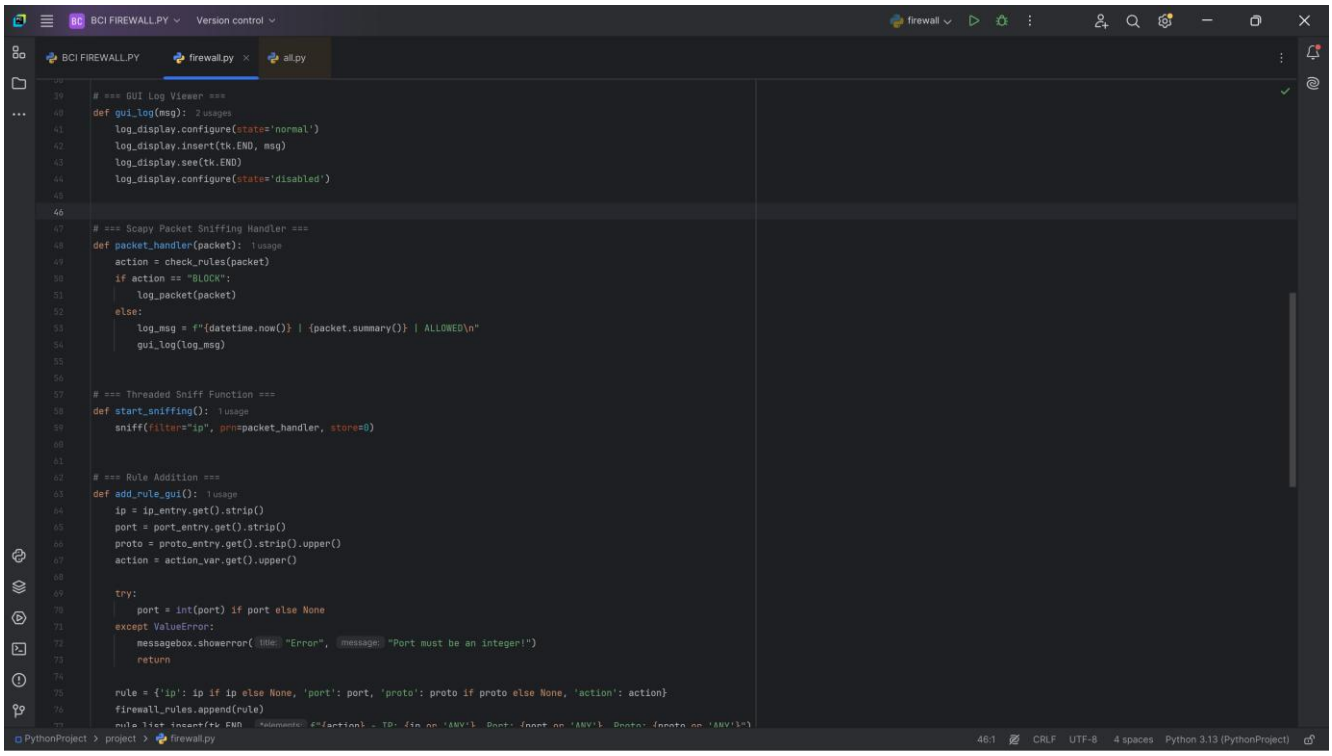




Final project: Personal Firewall using Python Report

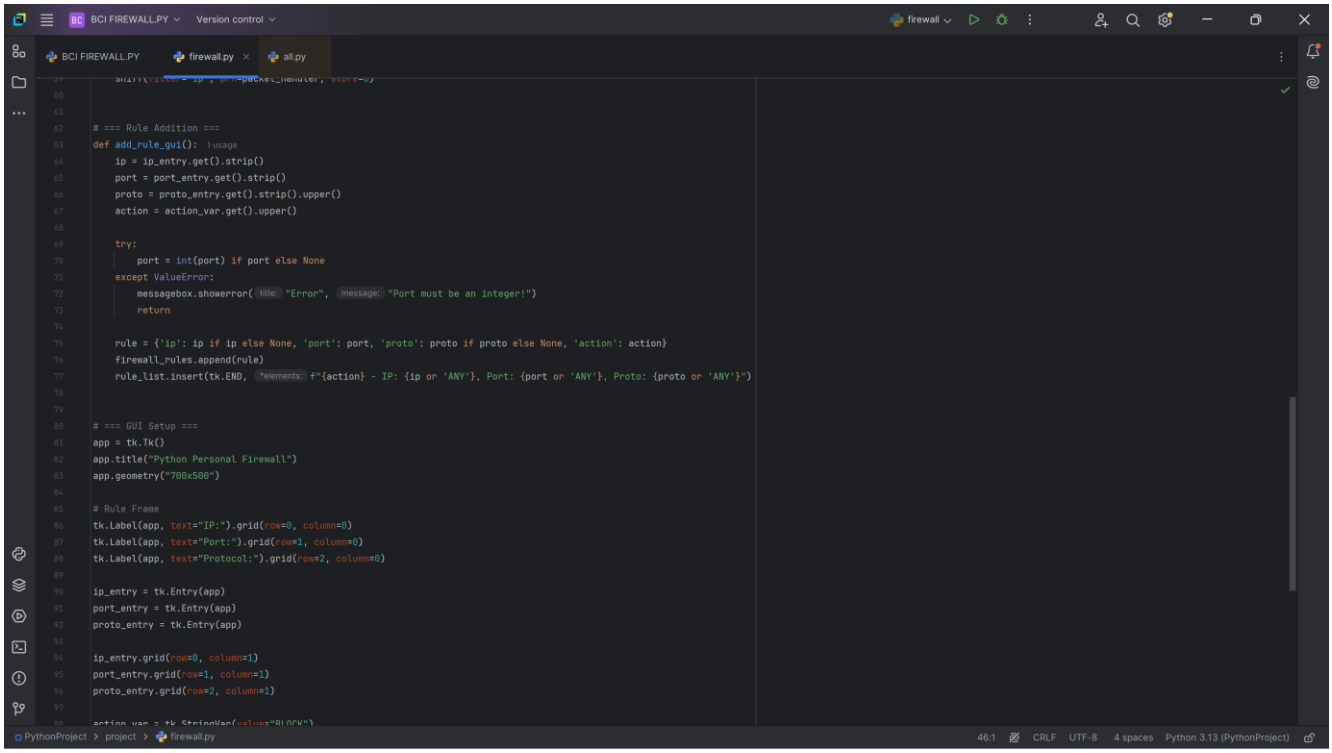


```python
import tkinter as tk
from tkinter import messagebox, scrolledtext
import threading
from scapy.all import sniff, IP, TCP, UDP
from datetime import datetime

# === Global Variables ===
firewall_rules = []
log_file = "firewall_log.txt"


# === Packet Rule Matching ===
def check_rules(packet):  1 usage
    if IP in packet:
        ip_src = packet[IP].src
        ip_dst = packet[IP].dst
        proto = "TCP" if packet.haslayer(TCP) else "UDP" if packet.haslayer(UDP) else "OTHER"
        port = packet[TCP].dport if packet.haslayer(TCP) else (packet[UDP].dport if packet.haslayer(UDP) else None)

        for rule in firewall_rules:
            if rule['ip'] and rule['ip'] not in (ip_src, ip_dst):
                continue
            if rule['port'] and rule['port'] != port:
                continue
            if rule['proto'] and rule['proto'] != proto:
                continue
            return rule['action']
    return "ALLOW"


# === Logging Function ===
def log_packet(packet, reason="BLOCKED"):  1 usage
    with open(log_file, "a") as f:
        log = f"{datetime.now()} | {packet.summary()} | {reason}\n"
        f.write(log)
        gui_log(log)


# === GUI Log Viewer ===
```

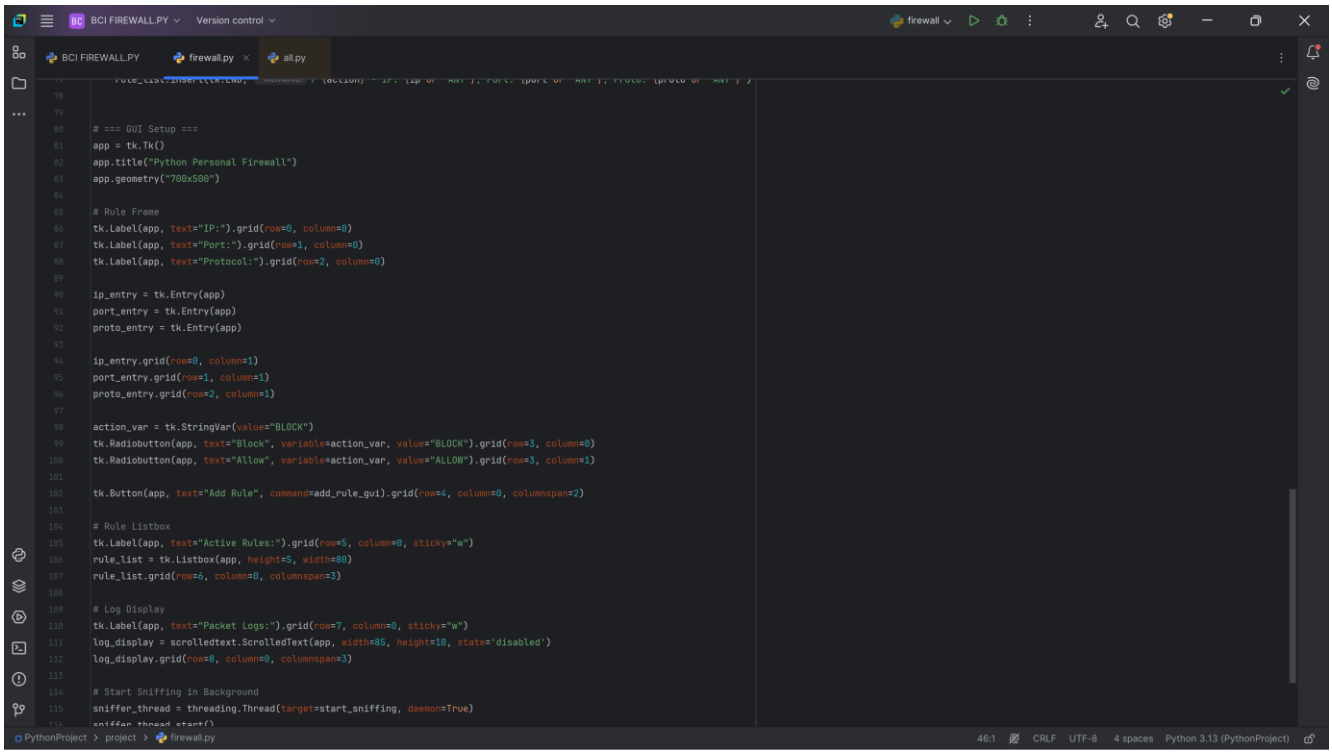


```python
# === GUI Log Viewer ===
def gui_log(msg):  2 usages
    log_display.configure(state='normal')
    log_display.insert(tk.END, msg)
    log_display.see(tk.END)
    log_display.configure(state='disabled')


# === Scapy Packet Sniffing Handler ===
def packet_handler(packet):  1 usage
    action = check_rules(packet)
    if action == "BLOCK":
        log_packet(packet)
    else:
        log_msg = f"{datetime.now()} | {packet.summary()} | ALLOWED\n"
        gui_log(log_msg)


# === Threaded Sniff Function ===
def start_sniffing():  1 usage
    sniff(filter="ip", prn=packet_handler, store=0)


# === Rule Addition ===
def add_rule_gui():  1 usage
    ip = ip_entry.get().strip()
    port = port_entry.get().strip()
    proto = proto_entry.get().strip().upper()
    action = action_var.get().upper()

    try:
        port = int(port) if port else None
    except ValueError:
        messagebox.showerror( title: "Error",  message: "Port must be an integer!")
        return

    rule = {'ip': ip if ip else None, 'port': port, 'proto': proto if proto else None, 'action': action}
    firewall_rules.append(rule)
```

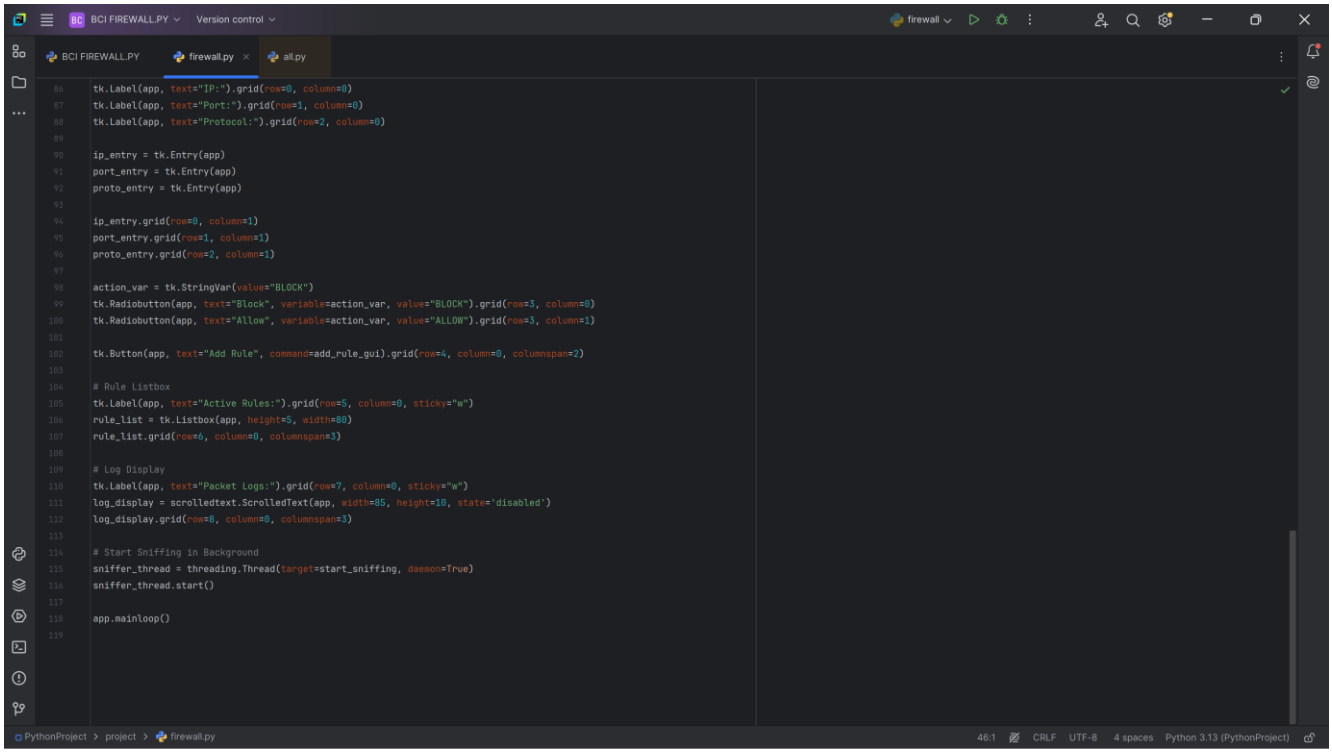
Final project: Personal Firewall using Python Report



```python
# === Rule Addition ===
def add_rule_gui():  1 usage
    ip = ip_entry.get().strip()
    port = port_entry.get().strip()
    proto = proto_entry.get().strip().upper()
    action = action_var.get().upper()

    try:
        port = int(port) if port else None
    except ValueError:
        messagebox.showerror( title: "Error",  message: "Port must be an integer!")
        return

    rule = {'ip': ip if ip else None, 'port': port, 'proto': proto if proto else None, 'action': action}
    firewall_rules.append(rule)
    rule_list.insert(tk.END,  *elements: f"{action} - IP: {ip or 'ANY'}, Port: {port or 'ANY'}, Proto: {proto or 'ANY'}")


# === GUI Setup ===
app = tk.Tk()
app.title("Python Personal Firewall")
app.geometry("700x500")

# Rule Frame
tk.Label(app, text="IP:").grid(row=0, column=0)
tk.Label(app, text="Port:").grid(row=1, column=0)
tk.Label(app, text="Protocol:").grid(row=2, column=0)

ip_entry = tk.Entry(app)
port_entry = tk.Entry(app)
proto_entry = tk.Entry(app)

ip_entry.grid(row=0, column=1)
port_entry.grid(row=1, column=1)
proto_entry.grid(row=2, column=1)
```

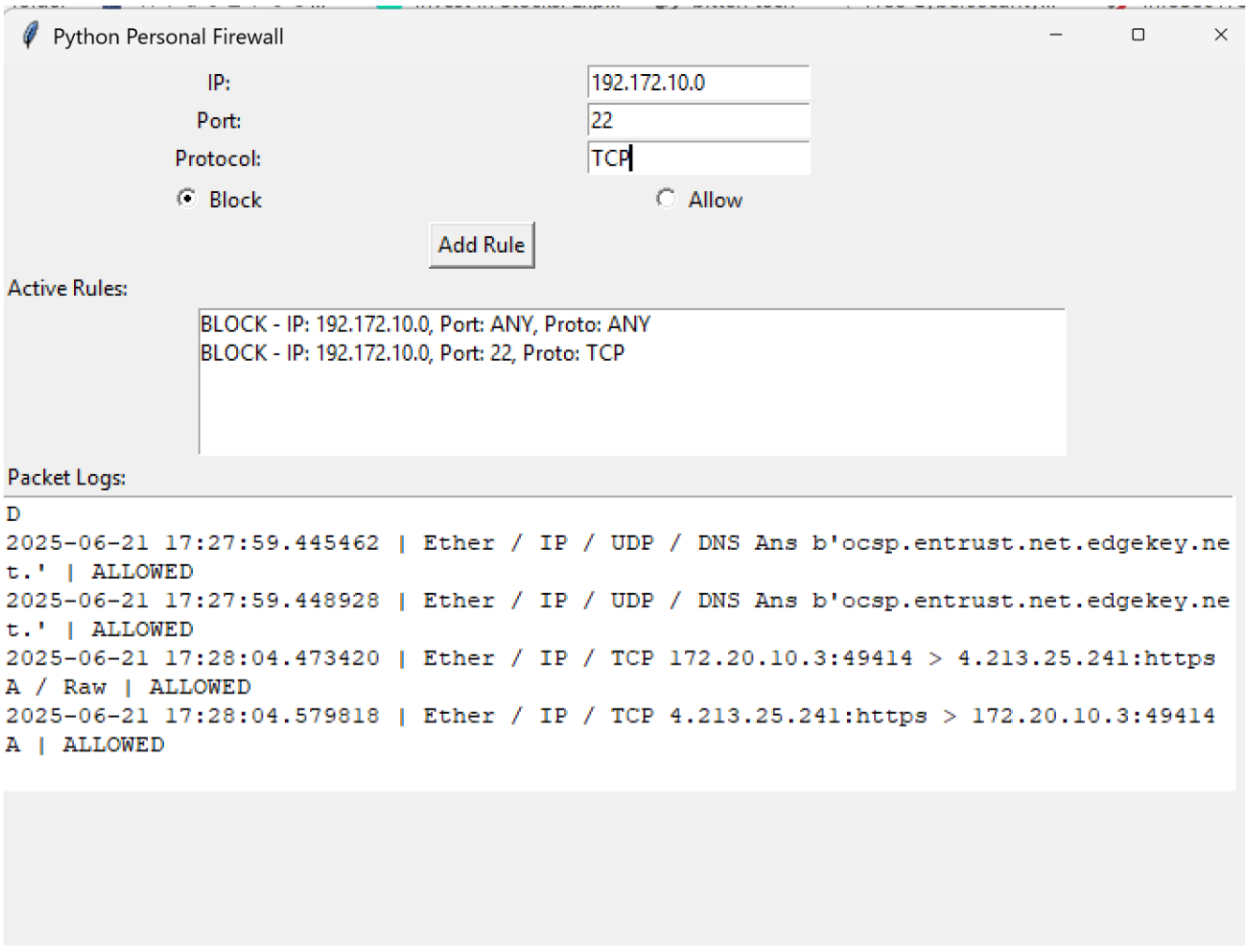


```python
# === GUI Setup ===
app = tk.Tk()
app.title("Python Personal Firewall")
app.geometry("700x500")

# Rule Frame
tk.Label(app, text="IP:").grid(row=0, column=0)
tk.Label(app, text="Port:").grid(row=1, column=0)
tk.Label(app, text="Protocol:").grid(row=2, column=0)

ip_entry = tk.Entry(app)
port_entry = tk.Entry(app)
proto_entry = tk.Entry(app)

ip_entry.grid(row=0, column=1)
port_entry.grid(row=1, column=1)
proto_entry.grid(row=2, column=1)

action_var = tk.StringVar(value="BLOCK")
tk.Radiobutton(app, text="Block", variable=action_var, value="BLOCK").grid(row=3, column=0)
tk.Radiobutton(app, text="Allow", variable=action_var, value="ALLOW").grid(row=3, column=1)

tk.Button(app, text="Add Rule", command=add_rule_gui).grid(row=4, column=0, columnspan=2)

# Rule Listbox
tk.Label(app, text="Active Rules:").grid(row=5, column=0, sticky="w")
rule_list = tk.Listbox(app, height=5, width=80)
rule_list.grid(row=6, column=0, columnspan=3)

# Log Display
tk.Label(app, text="Packet Logs:").grid(row=7, column=0, sticky="w")
log_display = scrolledtext.ScrolledText(app, width=85, height=10, state='disabled')
log_display.grid(row=8, column=0, columnspan=3)

# Start Sniffing in Background
sniffer_thread = threading.Thread(target=start_sniffing, daemon=True)
```



```
tk.Label(app, text="IP:").grid(row=0, column=0)
tk.Label(app, text="Port:").grid(row=1, column=0)
tk.Label(app, text="Protocol:").grid(row=2, column=0)

ip_entry = tk.Entry(app)
port_entry = tk.Entry(app)
proto_entry = tk.Entry(app)

ip_entry.grid(row=0, column=1)
port_entry.grid(row=1, column=1)
proto_entry.grid(row=2, column=1)

action_var = tk.StringVar(value="BLOCK")
tk.Radiobutton(app, text="Block", variable=action_var, value="BLOCK").grid(row=3, column=0)
tk.Radiobutton(app, text="Allow", variable=action_var, value="ALLOW").grid(row=3, column=1)

tk.Button(app, text="Add Rule", command=add_rule_gui).grid(row=4, column=0, columnspan=2)

# Rule Listbox
tk.Label(app, text="Active Rules:").grid(row=5, column=0, sticky="w")
rule_list = tk.Listbox(app, height=5, width=80)
rule_list.grid(row=6, column=0, columnspan=3)

# Log Display
tk.Label(app, text="Packet Logs:").grid(row=7, column=0, sticky="w")
log_display = scrolledtext.ScrolledText(app, width=85, height=10, state='disabled')
log_display.grid(row=8, column=0, columnspan=3)

# Start Sniffing in Background
sniffer_thread = threading.Thread(target=start_sniffing, daemon=True)
sniffer_thread.start()

app.mainloop()
```

FINAL RESULT



Final project: Personal Firewall using Python Report

Source code :-

1. **https://github.com/gantmama/project-firewall/pull/1/commits/08f2ac904516c7e80fcc630edaecc033fd4b9e33**

2.http://github.com/gantmama/project-firewall/blob/projects/scapy%5Call.py

Conclusion

This project provided hands-on experience in Python programming, packet analysis, and network security. The personal firewall can be a practical tool for basic traffic monitoring and rule-based filtering. It serves as an excellent foundation for further exploration into advanced firewall technologies and intrusion detection systems.